Assignment 2

Yahuan Shi, Armin Puran Youssef, Eckart Cobo-Briesewitz

2025.11.17

A. Forward Kinematics

1. Compute the DH parameters

i	α_{i-1}	a_{i-1}	d_i	θ_i
1	0	0	0	q_1
2	0	L_1	0	$q_2 - \pi/2$
3	0	L_2	0	q_3
4(E)	0	L_3	0	0

Table 1: DH parameters for the 3-DoF RRR Puma

2. Transformation between frames

The general DH transformation is:

$$ {}^{i-1}_{i}T = R_x(\alpha_{i-1})\, T_x(a_{i-1})\, R_z(\theta_i)\, T_z(d_i). $$

Homogeneous transformations matrix:

$$ {}^{i-1}_{i}T = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i \cos\alpha_{i-1} & \cos\theta_i \cos\alpha_{i-1} & -\sin\alpha_{i-1} & -\sin\alpha_{i-1} d_i \\ \sin\theta_i \sin\alpha_{i-1} & \cos\theta_i \sin\alpha_{i-1} & \cos\alpha_{i-1} & \cos\alpha_{i-1} d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{0}_{1}T = \begin{bmatrix} c_1 & -s_1 & 0 & 0 \\ s_1 & c_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{1}_{2}T = \begin{bmatrix} s_2 & c_2 & 0 & L_1 \\ -c_2 & s_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{2}_{3}T = \begin{bmatrix} c_3 & -s_3 & 0 & L_2 \\ s_3 & c_3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{3}_{E}T = \begin{bmatrix} 1 & 0 & 0 & L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

End-effector transform

$$^{0}_{E}T = {}^{0}_{1}T\,{}^{1}_{2}T\,{}^{2}_{3}T\,{}^{3}_{E}T$$

After simplification:

$$^{0}_{E}T = \begin{bmatrix} s_{123} & c_{123} & 0 & L_1c_1 + L_2s_{12} + L_3s_{123} \\ -c_{123} & s_{123} & 0 & L_1s_1 - L_2c_{12} - L_3c_{123} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where

$$s_{12} = s_1c_2 + c_1s_2, \quad c_{01} = c_1c_2 - s_1s_2,$$
$$s_{123} = s_{12}c_3 + c_{12}s_3, \quad c_{123} = c_3c_{12} - s_3s_{12}.$$

3. End effector position in Operational space

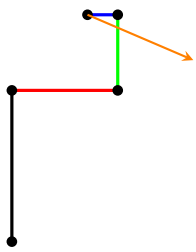
$$F(q) = \begin{bmatrix} L_1c_1 + L_2s_{12} + L_3s_{123} \\ L_1s_1 - L_2c_{12} - L_3c_{123} \\ q_1 + q_2 + q_3 - \frac{\pi}{2} \end{bmatrix}$$

4. Compute the End Effector Jacobian

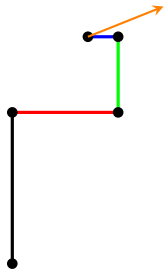
$$J(q) = \frac{\partial F(q)}{\partial q} = \begin{bmatrix} -L_1s_1 + L_2c_{12} + L_3c_{123} & L_2c_{12} + L_3c_{123} & L_3c_{123} \\ L_1c_1 + L_2s_{12} + L_3s_{123} & L_2s_{12} + L_3s_{123} & L_3s_{123} \\ 1 & 1 & 1 \end{bmatrix}$$

5. Understanding the Jacobian Matrix and Pose Singularities

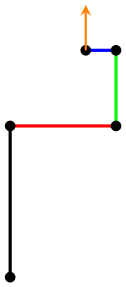
i) $q_1 = (0, 0, -\frac{\pi}{2})^T$



$\dot{q} = (1, 0, 0)$



$\dot{q} = (0, 1, 0)$

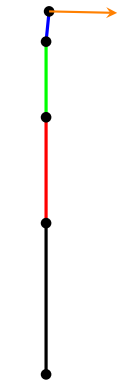


$\dot{q} = (0, 0, 1)$

The configuration is **not a singularity**, based on the direction and independence of the end-effector velocity vectors, it can move in x- and y-direction.

ii) $q_2 = (-\frac{\pi}{2}, \frac{\pi}{2}, 0.1)^T$

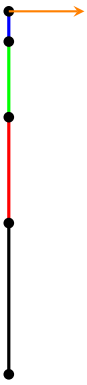
The robot is **close to a singularity**, the three velocity vectors become nearly parallel, so the end-effector can barely move in the y-direction.



$\dot{q} = (1, 0, 0)$

$\dot{q} = (0, 1, 0)$

$\dot{q} = (0, 0, 1)$



$\dot{q} = (1, 0, 0)$

$\dot{q} = (0, 1, 0)$

$\dot{q} = (0, 0, 1)$

iii) $q_3 = (-\frac{\pi}{2}, \frac{\pi}{2}, 0)^T$

The robot is **in a singularity** configuration, the robot can still move in the x-direction, but it has lost the ability to move in the y-direction.

B. Trajectory Generation in Joint Space

1.Generation of smooth trajectories with polynomial splines

The goal is to generate a smooth joint–space trajectory that starts at

$$q_a = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

passes through the via–point

$$q_b = \begin{bmatrix} -\frac{\pi}{4} \\ \frac{\pi}{2} \\ 0 \end{bmatrix} \quad \text{at } t = 2.5 \text{ s},$$

and reaches the final configuration

$$q_c = \begin{bmatrix} -\frac{\pi}{2} \\ \frac{\pi}{4} \\ 0 \end{bmatrix} \quad \text{at } t = 5 \text{ s}.$$

Each spline segment is defined as a cubic polynomial,

$$q_s(t) = a_0 + a_1(t - t_{start}) + a_2(t - t_{start})^2 + a_3(t_{start})^3,$$

with $t_{\text{start}} = 0$ and s as a parameter for the corresponding segment. Each q has it's own q_s.

From that the following conditions result:

$$q_0(0) = q_a \quad q_0(2.5) = q_b \quad \dot{q}_0(0) = 0$$

$$\dot{q}_0(2.5) = \begin{cases} 0, & \text{direction does change} \\ \text{Average v of each segment}, & \text{otherwise} \end{cases}$$

$$q_1(2.5) = q_b \quad q_1(5) = q_c$$

$$\dot{q}_1(2.5) = \begin{cases} 0, & \text{direction does change} \\ \text{Average v of each segment}, & \text{otherwise} \end{cases}$$

$$\dot{q}_0(5) = 0$$

That means, for each segments we have four conditions. This is a linear systems of equations. We can solve that with python and the library numpy!

```
    import numpy as np
from numpy import pi
import matplotlib.pyplot as plt

t_start = 0
t1 = 2.5
tf = 5.0

q_a = np.array([0.0, 0.0, 0.0])
q_b = np.array([-pi / 4, pi / 2, 0.0])
q_c = np.array([-pi / 2, pi / 4, 0.0])

v_avg_1 = (q_b - q_a) / t1
v_avg_2 = (q_c - q_b) / (tf - t1)

q_dot_a = np.zeros(3)
q_dot_b = np.zeros(3)
q_dot_c = np.zeros(3)

for i in range(3):
    v1 = v_avg_1[i]
    v2 = v_avg_2[i]

    if (v1 * v2 < 0):
        q_dot_b[i] = 0.0
    else:
        q_dot_b[i] = (v1 + v2) / 2
```

```
A_params = np.zeros((3, 2, 4))



# What is known: q(0), q(2,5), q'(0), q'(2,5) (first segment)

M_s = np.zeros((3, 2, 4, 4))
V_s = np.zeros((3,2,4))
S_s = np.zeros((3,2,4))

def q(t):
    # q(t)  = a1 + a2 * t + a3*t**2 + a4*t**3
    return np.array([1, t, t**2, t**3])

def q_dot(t):
    # q'(t) = 0 * a1 + a2 + 2*a3*t + 3*a4*t**2
    return np.array([0, 1, 2*t, 3*t**2])



for j in range(3):
    M_s[j][0] = np.array([q(0),q(2.5),q_dot(0),q_dot(2.5)])
    V_s[j][0] = np.array([q_a[j], q_b[j], q_dot_a[j],
       q_dot_b[j]])

for j in range(3):
    M_s[j][1] = np.array([q(2.5),q(5),q_dot(2.5),q_dot(5)])
    V_s[j][1] = np.array([q_b[j], q_c[j], q_dot_b[j],
       q_dot_c[j]])


# solve the equations:
for j in range(3):
    for s in range(2):
        M_s_inv = np.linalg.inv(M_s[j][s])
        S_s[j][s] = M_s_inv  @ V_s[j][s]
```

Here are the solution vectors for segment 1, each entry in the vector corresponds to a joint:

$$a_0 = \begin{pmatrix} 0.0 \\ 0.0 \\ 0.0 \end{pmatrix} \qquad a_1 = \begin{pmatrix} 1.40 \times 10^{-16} \\ -2.79 \times 10^{-16} \\ 0.0 \end{pmatrix}$$

$$a_2 = \begin{pmatrix} -0.25 \\ 0.75 \\ 0.0 \end{pmatrix} \qquad a_3 = \begin{pmatrix} 0.05 \\ -0.20 \\ 0.0 \end{pmatrix}$$

Here are the solution vectors for segment 2:

$$a_0 = \begin{pmatrix} -1.57 \\ -2.36 \\ 0.0 \end{pmatrix} \qquad a_1 = \begin{pmatrix} 1.26 \\ 3.77 \\ 0.0 \end{pmatrix}$$

$$a_2 = \begin{pmatrix} -0.50 \\ -1.13 \\ 0.0 \end{pmatrix} \qquad a_3 = \begin{pmatrix} 0.05 \\ 0.10 \\ 0.0 \end{pmatrix}$$

In 1 you can see q_t for each joint with respect to time.

2. Trajectories in joint space

In the following section we will explain how we make sure that v_{max} and a_{max} is not exceeded. Again we have for each joint one function q that is describing the trajectory of the joint over time. We also know the first two derivations.

$$q(t) = a_0 + a_1\, t + a_2\, t^2 + a_3\, t^3$$

$$\dot{q}(t) = a_1 + 2a_2\, t + 3a_3\, t^2$$

$$\ddot{q}(t) = 2a_2 + 6a_3\, t$$

However, we have again four conditions for our parameters $a_0, ..., a_3$.

$$\begin{aligned} q(t_s) &= q_0 \\ \dot{q}(t_s) &= 0 \\ q(t_f) &= q_f \\ \dot{q}(t_f) &= 0 \end{aligned}$$

When do equivalent transformation then we can get formulars for $a_0, ..., a_3$

$$a_0 = q_0, \quad a_1 = 0, \quad a_2 = \frac{3}{t_f^2}(q_f - q_0), \quad a_3 = -\frac{2}{t_f^3}(q_f - q_0)$$

Now we have to ask the question how to choose tf such that we are not exceeding v_{max} and a_{max}. In order to achieve that we are defining an additional condition:

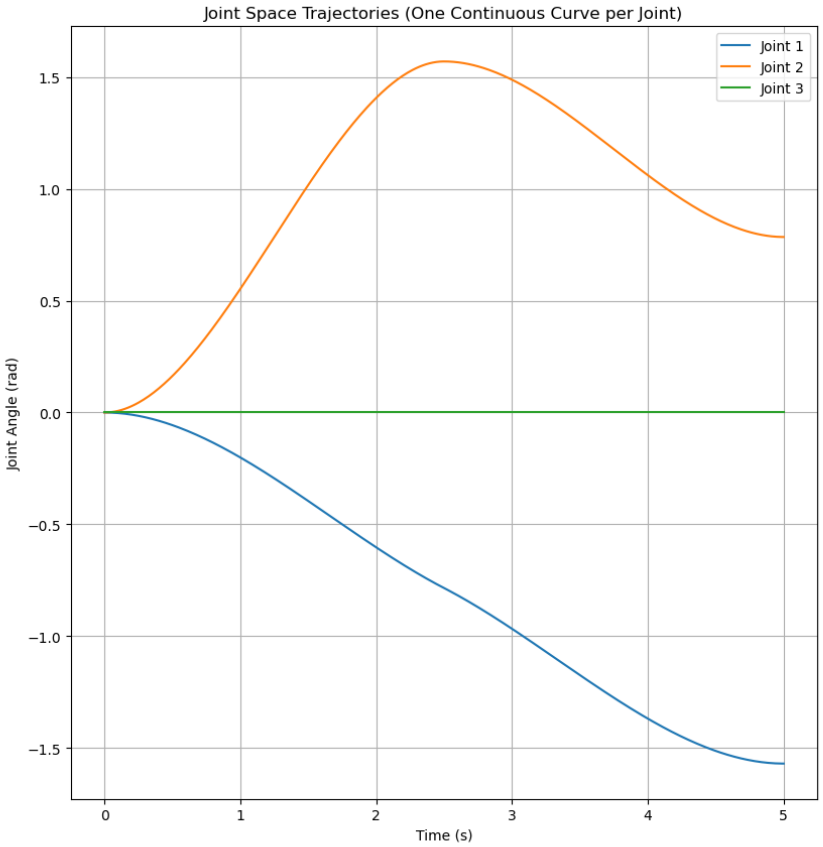


Figure 1: Graph of q_t for each joint with respect to time

$$|\dot{q}(t)| \leq v_{\max} \quad \wedge \quad |\ddot{q}(t)| < a_{\max}$$

Due to the fact that $\dot{q}(t)$ is a quadratic function and symmetric, we can follow that the global maximum of $\dot{q}(t)$ for $t \in [t_s, t_f]$ is at $t = t_f/2$. The function $\ddot{q}(t)$ is linear, thus the global maximum is at $t = 0$ and $t = t_f$ ($|\ddot{q}(0)| = |\ddot{q}(t_f)|$). When we do term forming we get:

$$\left|\frac{3}{2v_{\max}}\Delta q\right| \leq t_f \quad \wedge \quad \sqrt{\left|\frac{6\Delta q}{a_{\max}}\right|} \leq t_f$$

Where $\Delta q = q_f - q_0$. To satisfy both conditions, we just take the maximum value.

$$\max\left(\left|\frac{3}{2v_{\max}}\Delta q\right|, \sqrt{\left|\frac{6\Delta q}{a_{\max}}\right|}\right)$$

That must be done for every q(t) that corresponds to a joint. That is the idea of the code. However, in order to make sure that every joint is done with its movement at the same time, we have to take the largest t_f over all joints.

In 4, 3 and 2 you can see the plots for execution the proj1control function. The chosen gains are the following:

$$\mathbf{k}_p = \begin{pmatrix} 400 \\ 400 \\ 400 \end{pmatrix}, \quad \mathbf{k}_v = \begin{pmatrix} 40 \\ 40 \\ 40 \end{pmatrix}$$

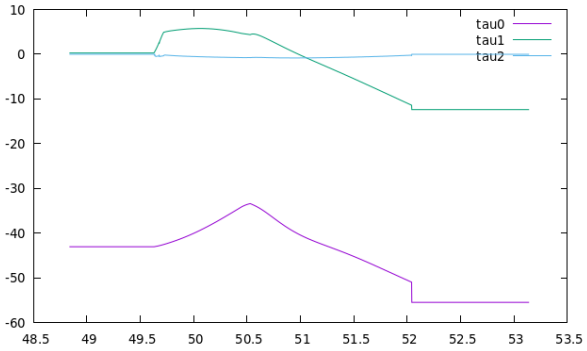


Figure 2: τ for each joint with respect to time

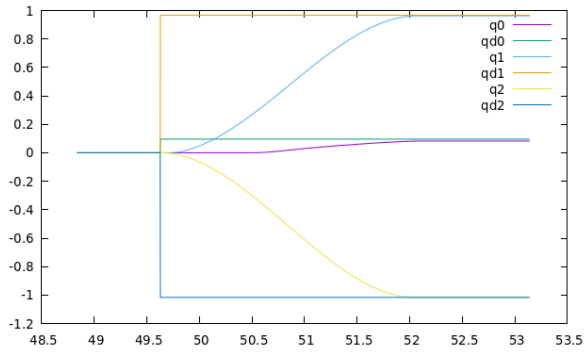


Figure 3: q for each joint with respect to time

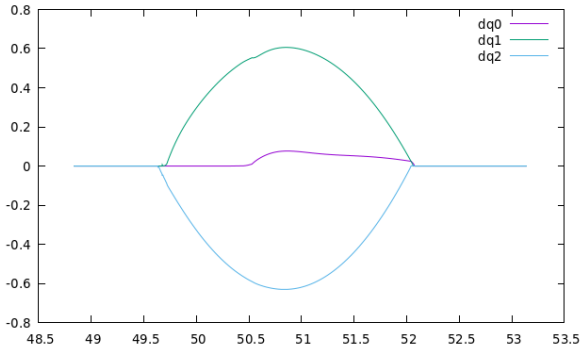


Figure 4: *dq* for each joint with respect to time

C. Operational Space Control

1.Project 2 - Circle

control.cpp

2.Project 2 - Control

control.cpp

3.Project 2 - Gain tuning and plots

Figures 5 and 6 show different values for the circle trajectory with the tuned controller. For higher Kp the trajectory is tracked better, so $x - x_d$ shows lower values, and the spike in error at the start of the trajectory looks more pointy. In addition, there is a higher output force τ at the beginning of the trajectory since the reference starts at a high velocity directly. For lower Kp, the spike in the error at the beginning of the trajectory is smoother but the average tracking error is higher.

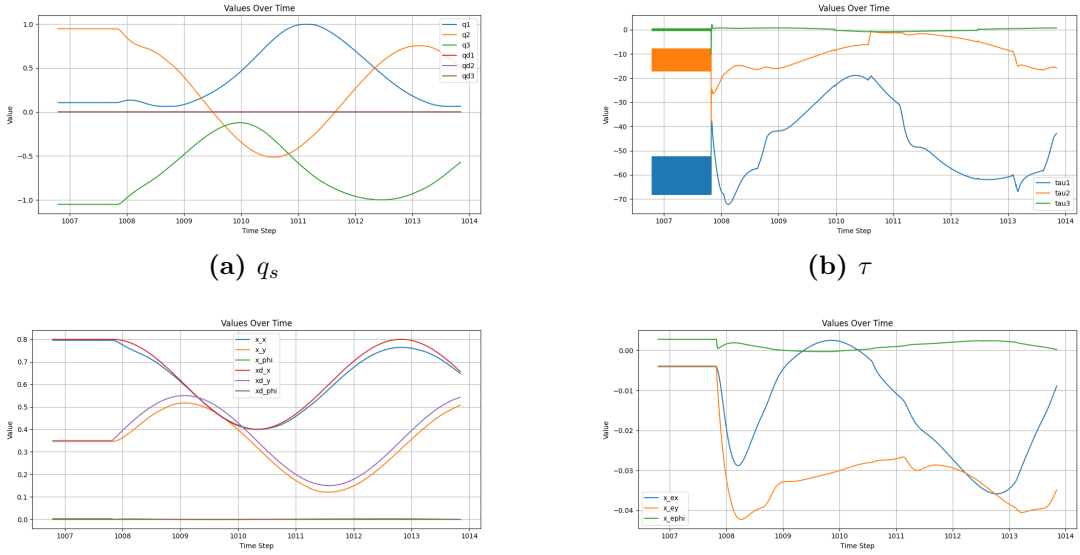
4.Project 3 - Parabolic Blends

Figure 7 shows the desired β and $\dot{\beta}$ values for the circle trajectory using the parabolic blends method. This method ensures that the reference trajectory slowly increases the desired velocity instead of setting a high velocity directly, which is physically impossible.

5.Optional

Contributions:

Student Name	**A1**	**A2**	**A3**	**A4**	**A5**	**B1**	**B2**	**C1**	**C2**	**C3**	**C4**	**C5**
Yahuan Shi	x	x	x	x	x	x						
Armin Puran Youssef						x	x					
Eckart Cobo-Briesewitz								x	x	x	x	



(a) q_s **(b)** τ

(c) x **(d)** $x - x_d$

Figure 5: Kp: 5000, 5000, 4000, Kd: 400, 400, 40

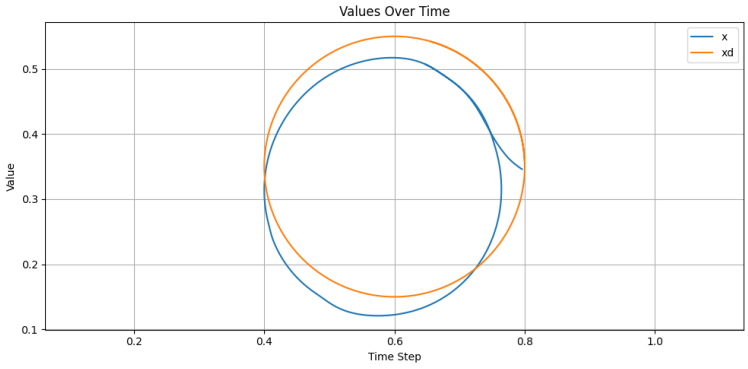


Figure 6: XY-plane visualization of the desired end-effector trajectory and the real trajectory. Kp: 5000, 5000, 4000, Kd: 400, 400, 40

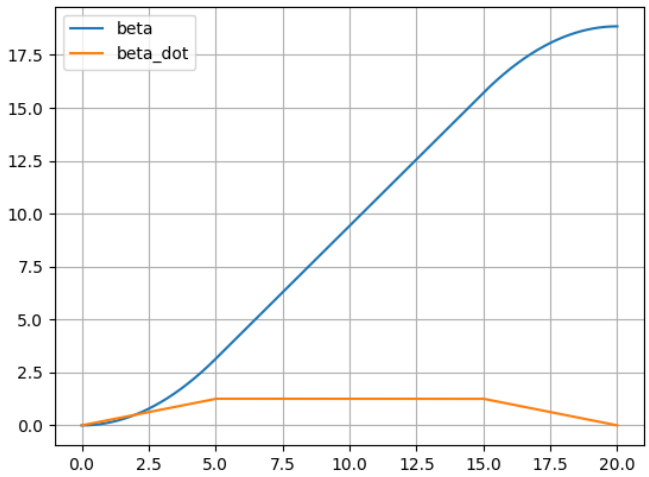


Figure 7: Desired β and $\dot{\beta}$ for the Parabolic Blends method.